Prueba tecnica Java Backend: modernizacion de Spring PetClinic con IA

Prueba para candidatos Java Backend (Semi Senior / Senior)
Repositorio de origen: https://github.com/spring-projects/spring-petclinic
Lenguaje minimo requerido: Java 21
Tiempo de ejecucion: 1 (un) dia
Alcance obligatorio: backend y APIs REST
Frontend: opcional y no requerido para aprobar
Uso de herramientas de IA: obligatorio, critico y documentado

1. Contexto y objetivo

El ejercicio consiste en analizar y modernizar la aplicacion **Spring PetClinic** suministrada como codigo de partida, mediante una migracion hacia una solucion backend moderna, mantenible, modular y documentada.

El equipo evaluador ha **compilado y ejecutado funcionalmente la version original**, por lo que esta constituye la **linea base de comportamiento**. El candidato debe preservar sus funcionalidades y reglas de negocio en el alcance migrado. No basta con generar un proyecto nuevo sin relacion verificable con el codigo original.

El objetivo principal es evaluar la capacidad de **analizar codigo existente, identificar tecnologias y validaciones, proponer una estrategia de migracion, implementar la solucion y utilizar IA con criterio tecnico**. Tambien se evaluaran buenas practicas de desarrollo, pruebas, documentacion, versionamiento y reproducibilidad.

Tiempo disponible

La prueba tecnica tendra una duracion total de **1 (un) dia**. Dentro de ese plazo, el candidato debera realizar el analisis, la propuesta de arquitectura, la implementacion, las pruebas, la documentacion y la publicacion de su trabajo en un **repositorio publico propio**. Se recomienda priorizar una solucion backend funcional, verificable y bien documentada frente a incorporar componentes opcionales.

2. Objetivos especificos

1. **Analisis de codigo:** identificar componentes, flujos, dependencias, acoplamiento, deuda tecnica y reglas de negocio del proyecto original.
2. **Tecnologias:** reconocer versiones, frameworks, librerias y mecanismos de persistencia presentes; justificar las decisiones de actualizacion.
3. **Validaciones:** localizar, preservar y probar validaciones y casos limite del sistema legacy.
4. **Estrategia de migracion:** definir limites de modulos/servicios y una secuencia de transformacion realista que reduzca riesgos.

5. **Implementacion y uso de IA:** aprovechar herramientas de IA para investigar, diseñar, implementar, probar y documentar, revisando criticamente sus resultados.
6. **Calidad:** aplicar separacion de responsabilidades, buenas practicas Java, seguridad basica, manejo de errores, pruebas y trazabilidad.

3. Reglas de participacion y entrega (obligatorias)

MUY IMPORTANTE: CADA CANDIDATO DEBE PUBLICAR TODO SU TRABAJO EN UN REPOSITORIO PUBLICO PROPIO, BAJO SU AUTORIA.

- Clonar el repositorio base: **https://github.com/spring-projects/spring-petclinic**.
- Crear un **nuevo repositorio publico bajo su propia cuenta de GitHub**, con permisos para que el equipo evaluador lo consulte.
- Publicar en ese repositorio **todo lo implementado**: codigo fuente, modulos o microservicios, configuracion de ejemplo, scripts de base de datos, tests, documentacion, diagramas, colecciones de pruebas y bitacora de IA.
- Conservar **el historial Git del proyecto base** y añadir un historial propio, progresivo y verificable de commits. No entregar unicamente un snapshot del codigo final.
- Trabajar con ramas `main`, `develop` y al menos una `feature/*`, con integraciones visibles y ordenadas. Las ramas `release/*` y `hotfix/*` son opcionales cuando exista una necesidad real.
- Entregar la **URL del repositorio propio**, el **commit o tag de la version final** y una breve indicacion de las ramas principales utilizadas.
- No se aceptan entregas exclusivamente en archivos ZIP, enlaces a carpetas locales o repositorios que contengan solo binarios.
- No incluir credenciales reales, tokens, secretos, archivos `.env` con datos sensibles ni bases de datos privadas.

La autoria del repositorio de entrega no implica reclamar la autoria del codigo original: deben conservarse los avisos de licencia y atribucion que correspondan al proyecto de origen.

3.1. Ejemplo de preparacion del repositorio propio

Estos comandos son una guia; se acepta una estrategia equivalente que **preserve la historia original**:

```
# 1) Clonar el repositorio original (con su historial)
git clone https://github.com/spring-projects/spring-petclinic.git
cd spring-petclinic

# 2) Conservar el origen como referencia de solo lectura
git remote rename origin upstream

# 3) Crear un repositorio publico VACIO bajo tu cuenta en GitHub.
# No inicializarlo con otro README, .gitignore ni licencia.
```

```
# Sustituir TU_USUARIO y TU_REPOSITORIO por los propios.
git remote add origin https://github.com/TU_USUARIO/TU_REPOSITORIO.git

# 4) Confirmar cual es la rama principal local
git branch --show-current

# Si la rama local principal no se llama main, normalizarla:
# git branch -m main

# 5) Publicar la rama principal, conservando los commits existentes
git push -u origin main

# 6) Crear la rama de integracion
git switch -c develop
git push -u origin develop

# 7) Trabajar sobre una rama de funcionalidad
git switch -c feature/owners-api
git push -u origin feature/owners-api
```

> **Atencion:** el comando `git push -u origin main` supone que la rama local se llama `main`; verificarlo previamente. No hacer `push` sobre `upstream` ni modificar el repositorio original. Evitar `force push`, `squash` de toda la historia o acciones que oculten la evolucion del trabajo.

4. Alcance tecnico y funcional

4.1. Obligatorio

- **Java 21 o superior**, con compilacion y ejecucion documentadas.
- Spring Boot y dependencias compatibles con Java 21; justificar las versiones seleccionadas.
- **Backend REST/JSON** que exponga las capacidades funcionales migradas. No es obligatorio conservar las vistas MVC/Thymeleaf.
- **Separacion en al menos dos modulos funcionales de negocio**, con limites y dependencias explicitos. Se acepta un **monolito modular bien delimitado** o una solucion de **microservicios desplegables por separado**, siempre que la eleccion este debidamente argumentada.
- Conservar en la capa backend las capacidades originales relevantes de **propietarios, mascotas, visitas y veterinarios**; documentar las correspondencias entre operaciones originales y APIs nuevas.
- Conservar y probar reglas de negocio y validaciones existentes. Cualquier desviacion intencional debe estar expresamente identificada y justificada.
- Definir contratos HTTP consistentes: rutas, metodos, DTO, validacion de entrada, codigos de estado y respuestas de error.
- **Documentacion OpenAPI/Swagger** accesible para las APIs desarrolladas.

- **Pruebas unitarias y de integracion**, incluyendo flujos exitosos, entradas invalidas y casos limite.
- **README de puesta en marcha y pruebas** que permita ejecutar la solucion desde un entorno limpio.
- Historial Git y bitacora de uso de IA conforme a este documento.

4.2. Opcional / valor agregado

- Frontend nuevo o adaptado (no requerido y sin prioridad frente al backend).
- Docker Compose para levantar la solucion y sus dependencias.
- CI (por ejemplo, GitHub Actions) para compilacion, tests y verificaciones de calidad.
- Observabilidad basica (logs estructurados, correlation ID, health checks y metricas).
- Pruebas de contrato, cobertura automatizada y herramientas de analisis estatico.

No se exige agregar Kafka, Kubernetes, un API Gateway ni componentes distribuidos sin necesidad. No se premia multiplicar servicios de manera artificial.

5. Arquitectura recomendada

Se espera una **separacion por capacidades de negocio**, no necesariamente un servicio por entidad. Una posible organizacion de referencia es:

- **Owners / Pets:** gestion de propietarios y sus mascotas (dominio fuertemente relacionado).
- **Visits:** registro y consulta de visitas asociadas a mascotas.
- **Vets:** consulta de veterinarios y especialidades.

Se pueden proponer otros limites si se explican las dependencias, la propiedad de datos y los compromisos de consistencia. En el caso de microservicios, cada servicio debe tener una responsabilidad clara, contratos explicitos y evitar el acceso directo a tablas propiedad de otro servicio sin justificacion.

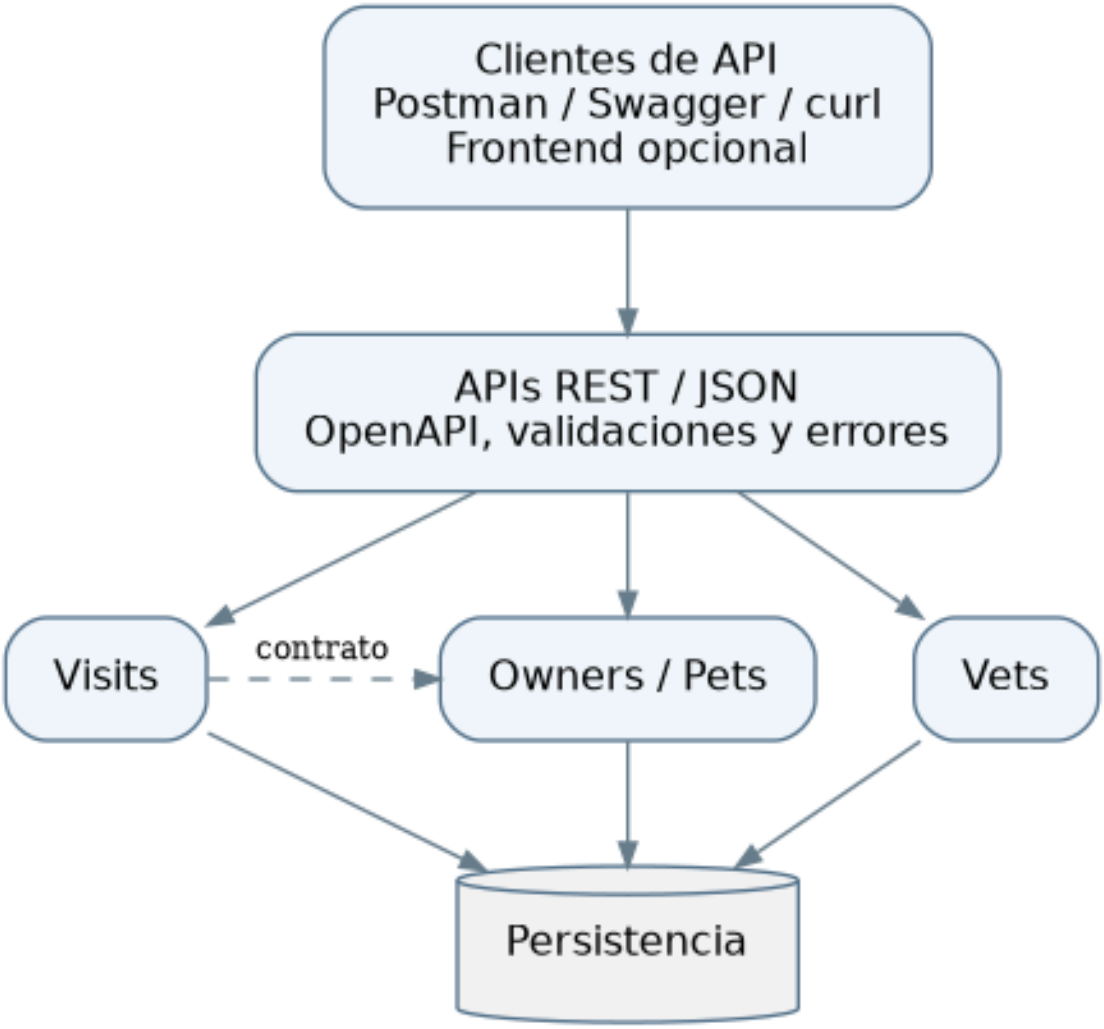


Diagrama 1

> **Nota:** el esquema representa **responsabilidades logicas**. La base de datos compartida se ilustra como una simplificacion para un monolito modular o una transicion; **no es una recomendacion de compartir indiscriminadamente el mismo esquema entre microservicios**. Si se eligen microservicios independientes, explicar propiedad de datos, integracion entre servicios, transacciones y consistencia.

5.1. Estructura interna sugerida

En cada modulo o servicio se recomienda diferenciar:

```
api/              # Controllers REST, DTO, validacion, manejo de errores
application/      # Casos de uso y coordinacion de operaciones
domain/           # Modelo y reglas de negocio
infrastructure/   # Persistencia, adaptadores, clientes externos y
configuracion
test/             # Pruebas correspondientes (estructura segun framework)
```

Es un ejemplo, no una estructura prescriptiva. Se evaluara la **claridad de las responsabilidades** y no la coincidencia exacta de nombres de carpetas. Evitar exponer directamente entidades de persistencia como contrato publico cuando ello genere acoplamiento.

5.2. Recomendaciones de diseño

- Utilizar DTO de solicitud y respuesta; separar contratos externos de entidades JPA.
- Centralizar manejo de excepciones y errores HTTP, con mensajes comprensibles y sin filtrar detalles internos.
- Aplicar Bean Validation y conservar validaciones de negocio en la capa correspondiente.
- Evitar logica de negocio en controladores y consultas de persistencia dispersas.
- Gestionar la configuracion mediante perfiles y variables de entorno documentadas.
- Definir politicas de paginacion, filtros y respuesta para consultas.
- Evitar introducir mecanismos de autenticacion, infraestructura y comunicacion distribuidas que no aporten al alcance de la prueba.
- Documentar decisiones relevantes y alternativas descartadas, incluyendo por que se opto por monolito modular o microservicios.

6. Proceso de trabajo esperado

Fase A. Diagnostico

Antes de modificar el codigo, registrar:

- Tecnologias y versiones encontradas.
- Componentes y dependencias principales.
- Funcionalidades y endpoints/vistas actuales relacionados con propietarios, mascotas, visitas y veterinarios.
- Validaciones, reglas de negocio y casos limite identificados.
- Riesgos de migracion, deuda tecnica y elementos que se mantendran o reemplazaran.

Entregable sugerido: `docs/01-diagnostico-legacy.md`.

Fase B. Estrategia de migracion

Definir:

- Arquitectura objetivo, limites de modulos/servicios y sus responsabilidades.
- Plan incremental de migracion y validacion.
- Correspondencia entre casos de uso originales y nuevos endpoints.
- Estrategia de acceso a datos, contratos y gestion de errores.
- Decisiones tecnicas y sus motivos.

Entregables sugeridos: `docs/02-arquitectura.md`, `docs/03-plan-migracion.md` y diagramas Mermaid o imagenes.

Fase C. Implementacion

- Actualizar a Java 21 o superior.
- Implementar los modulos o microservicios definidos.
- Exponer APIs REST y documentarlas mediante OpenAPI.
- Migrar casos de uso y validaciones sin perder funcionalidad.
- Incorporar pruebas y configuracion reproducible.

Fase D. Validacion y entrega

- Ejecutar compilacion y tests desde un clon limpio del repositorio propio.
- Demostrar al menos los flujos de propietarios, mascotas, visitas y veterinarios.
- Documentar rutas y ejemplos para ejecutar llamadas a las APIs.
- Integrar cambios a `develop`, preparar la version final en `main` y registrar el commit o tag de entrega.
- Verificar que la bitacora de IA, la arquitectura y las instrucciones de instalacion esten publicadas.

7. Documentacion obligatoria

El repositorio del candidato debe incluir como minimo:

Documento o recurso	Contenido requerido
`README.md`	Objetivo, requisitos, arquitectura, instalacion, ejecucion y pruebas paso a paso
`docs/01-diagnostico-legacy.md`	Hallazgos tecnicos y funcionales del proyecto original
`docs/02-arquitectura.md`	Diagramas, limites, integraciones y decisiones arquitectonicas
`docs/03-plan-migracion.md`	Alcance migrado, secuencia, riesgos y matriz original vs. nuevo
`docs/04-uso-ia.md`	Herramientas, prompts representativos, resultados y validaciones humanas
OpenAPI	Especificacion de todos los endpoints implementados y ruta para visualizarla
Pruebas	Codigo de tests y comandos de ejecucion; ejemplos Postman, curl o HTTP
Configuracion de ejemplo	Variables no sensibles y pasos de inicializacion de base de datos

Documentar el codigo con comentarios o JavaDoc **cuando expliquen decisiones, contratos o reglas no evidentes**; evitar comentarios redundantes que solo repitan lo que hace cada linea. Los ejemplos de solicitudes y respuestas deben ser utilizables.

8. Guia minima de puesta en marcha (que debe completar cada candidato)

El candidato debe sustituir los marcadores siguientes por comandos, archivos y rutas REALES de su implementacion. No basta con copiar esta plantilla.

8.1. Prerrequisitos

Indicar las versiones concretas necesarias de:

- JDK (minimo 21).
- Maven o Gradle (o explicar como utilizar el wrapper incluido).
- Motor de base de datos, si aplica.
- Docker y Docker Compose, solo si la solucion los requiere.

8.2. Clonar su solucion

```
git clone https://github.com/TU_USUARIO/TU_REPOSITORIO.git
cd TU_REPOSITORIO
```

8.3. Configurar ambiente

Detallar:

1. Archivos de ejemplo (`.env.example`, `application-local.yml`, etc.).
2. Variables de entorno obligatorias, valores permitidos y valores de ejemplo **no sensibles**.
3. Motor de BD, creacion del esquema, migraciones y datos de prueba.
4. Puertos de escucha de cada servicio y dependencias que deben iniciarse antes.
5. Perfiles de Spring utilizados y sus propositos.

8.4. Compilar, ejecutar y verificar

Incluir los **comandos exactos** usados en el repositorio para:

```
[COMANDO REAL PARA COMPILAR]
[COMANDO REAL PARA EJECUTAR PRUEBAS]
[COMANDO REAL PARA INICIAR MODULOS/SERVICIOS]
[URL REAL DE HEALTH CHECK O VERIFICACION DE ARRANQUE]
[URL REAL DE SWAGGER UI Y/O OPENAPI JSON]
```

8.5. Probar funcionalmente

Explicar paso a paso **como probar cada capacidad migrada**, indicando:

- Endpoint, metodo HTTP, parametros y cuerpo JSON.
- Datos de prueba o prerequisitos necesarios.
- Respuesta esperada y codigo HTTP.
- Casos de error/validacion esperados.
- Como repetir o restablecer los datos para que las pruebas sean reproducibles.

Como minimo, incluir escenarios para **crear/consultar/actualizar propietarios y mascotas segun el comportamiento original, registrar y consultar visitas y consultar veterinarios**. La equivalencia funcional debe aclararse en la matriz de migracion.

Condicion de reproducibilidad: el evaluador debe poder partir de un entorno limpio, clonar **solo el repositorio publico del candidato**, montar las dependencias y verificar la solucion siguiendo exclusivamente la documentacion entregada.

9. GitFlow y calidad del historial

Se requiere una estrategia **basada en GitFlow**, proporcional al ejercicio:

- `main`: version estable y entregable.
- `develop`: integracion continua de cambios.
- `feature/*`: desarrollo de capacidades o tareas aisladas.
- `release/*`: preparacion de entrega, **si corresponde**.
- `hotfix/*`: correccion urgente, **si corresponde**.

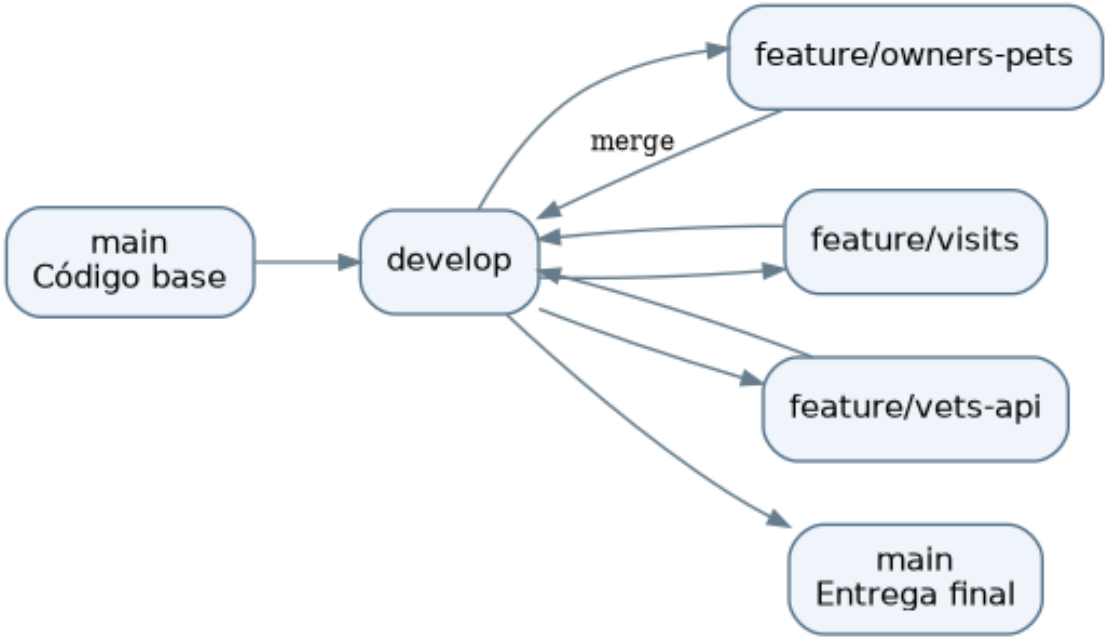


Diagrama 2

Reglas de evaluacion:

- Commits frecuentes, coherentes y con mensajes descriptivos; se recomienda *Conventional Commits* (por ejemplo, `feat: expose owners REST endpoints`, `test: cover visit validation`, `docs: explain local setup`).
- Cambios trazables por rama y, cuando sea posible, por Pull Request; documentar la integracion si se trabaja individualmente.
- Evitar un unico commit de “entrega final”, reescribir la historia o borrar ramas sin motivo.

- No es obligatorio utilizar todas las ramas de GitFlow si no existe una situacion real que las justifique.
- Se evaluara la **calidad del flujo**, no simplemente la cantidad de commits o ramas.

10. Uso obligatorio y responsable de IA

Se permite y **se exige** utilizar herramientas de IA (por ejemplo, ChatGPT, Claude, GitHub Copilot, Cursor u otras) para apoyar la solucion. No se medira la cantidad de codigo generado ni la cantidad de prompts.

La bitacora `docs/04-uso-ia.md` debe indicar, al menos:

1. Herramientas y versiones/modelos utilizados, cuando esten disponibles.
2. Objetivo de cada uso relevante (analisis, diseño, refactorizacion, tests, documentacion).
3. Prompts representativos o resumen verificable de instrucciones dadas.
4. Sugerencias aceptadas, rechazadas o corregidas y sus razones.
5. Como se valido tecnicamente el resultado: revision de codigo, tests, ejecucion o comparacion con reglas originales.
6. Limitaciones o errores de IA detectados.

La responsabilidad del resultado es del candidato. No se aceptara como justificacion de una implementacion incorrecta que "la IA lo propuso". Tampoco se deben publicar secretos ni informacion privada en prompts o documentacion.

11. Pruebas y criterios de calidad

Se espera cobertura razonada de:

- Casos de uso principales del codigo original.
- Validaciones de datos y reglas de negocio.
- Respuestas HTTP correctas y manejo de errores.
- Persistencia y consultas cuando corresponda.
- Casos limite, inexistencia de registros, entradas invalidas y conflictos.

Utilizar JUnit 5, Mockito y pruebas de integracion compatibles con el stack elegido. La cifra de cobertura, si se reporta, debe acompañarse de explicacion del alcance: **una cobertura alta no sustituye pruebas relevantes**.

Se valorara positivamente separar pruebas unitarias de integracion y ofrecer un comando unico que el evaluador pueda ejecutar desde un entorno limpio.

12. Criterios de evaluacion (100 puntos)

Criterio	Puntaje
Analisis y comprension del codigo legacy	15
Arquitectura y estrategia de migracion	15

Criterio	Puntaje
Implementacion y buenas practicas Java	25
Pruebas y preservacion funcional	15
Uso estrategico y validacion de IA	15
Documentacion tecnica, API y ambiente	10
GitFlow, historial y calidad de commits	5
Total	**100**

Referencia de aprobacion: 70/100, siempre que se cumplan los requisitos criticos. El evaluador puede realizar una defensa tecnica, solicitar la demostracion de casos de uso y pedir que se explique una decision de implementacion o una correccion de IA.

12.1. Condiciones criticas de aceptacion

- El codigo entregado **compila y se ejecuta con Java 21 o superior**.
- La solucion backend es funcional y su alcance esta explicado.
- Existen **al menos dos modulos funcionales** claramente separados.
- Las reglas de negocio del alcance migrado no se eliminan silenciosamente.
- Las APIs REST tienen documentacion OpenAPI.
- Se incluyen pruebas automatizadas ejecutables.
- Se documenta como instalar, configurar, iniciar y probar el ambiente.
- **Todo el trabajo esta publicado en un repositorio PUBLICO PROPIO** del candidato, accesible y con historial Git verificable.
- Se documenta el uso de IA y el proceso de verificacion humana.

La ausencia de un requisito critico, como un proyecto que no compila, la imposibilidad de reproducir su ejecucion o la falta de repositorio propio, puede impedir la aprobacion con independencia del puntaje parcial.

13. Entrega final: lista de verificacion

Antes de compartir su solucion, el candidato debe comprobar:

- ☐ Mi repositorio **es publico y esta bajo mi cuenta de GitHub**.
- ☐ El repositorio contiene **todo mi trabajo implementado**, no solo un ZIP o una compilacion.
- ☐ He conservado la trazabilidad del codigo original y mi historial de commits.
- ☐ Existen `main`, `develop` y al menos una rama `feature/*` con un flujo coherente.
- ☐ La version final esta integrada a `main` y tiene commit o tag identificable.
- ☐ El proyecto utiliza Java 21+ y los pasos de compilacion funcionan.
- ☐ Los modulos/servicios y sus responsabilidades estan documentados.
- ☐ Las APIs REST tienen OpenAPI y ejemplos de uso.
- ☐ La instalacion y la configuracion de base de datos estan documentadas.

- ☐ He indicado como ejecutar la solucion y cada conjunto de pruebas.
- ☐ He preservado y probado las funcionalidades de negocio migradas.
- ☐ Mi bitacora de IA documenta decisiones, verificaciones y errores detectados.
- ☐ No he publicado credenciales, tokens ni informacion sensible.
- ☐ El evaluador puede reproducir la solucion desde un clon limpio.

Formato de comunicacion de la entrega

```
Nombre del candidato: [NOMBRE]
Repositorio publico propio: https://github.com/[USUARIO]/[REPOSITORIO]
Rama de entrega: main
Commit SHA o tag final: [IDENTIFICADOR]
Ruta de instrucciones: README.md
Ruta de documentacion de arquitectura: docs/02-arquitectura.md
Ruta de bitacora IA: docs/04-uso-ia.md
Observaciones / limitaciones: [DETALLE]
```

Mensaje final al candidato: valoramos mas una migracion coherente, probada, explicada y con decisiones tecnicas justificadas que una arquitectura artificialmente compleja. El uso de IA forma parte del ejercicio, pero el razonamiento, la validacion y la calidad de la entrega son responsabilidad de quien desarrolla la solucion. **El trabajo debe estar completo y publicado en un repositorio publico de su propia autoria.**